# Informe: Prim vs. Kruskal

Taller 6 – Ciencias de la Computación 2


| Nombre | Código |
|---|---|
| Jose Fernando Ramirez Ortiz | 20241020080 |
| Joel David Pérez Arroyave | 20242020017 |
| Carlos Alberto Barriga Gamez | 20222020179 |
| Miguel Ángel Hernández Medina | 20222020035 |


## 1. Introducción

Prim y Kruskal calculan el **árbol de expansión mínima** de un grafo no dirigido y ponderado: el conjunto de relaciones que conecta todos los vértices, sin ciclos y con el menor peso total. Con $V$ vértices, el árbol siempre tiene $V - 1$ relaciones. Este informe explica brevemente cómo se implementaron y los compara.

## 2. Implementación

Cada algoritmo está en su propia clase (`Prim` y `Kruskal`) y tiene dos métodos: `ejecutar`, que imprime el resultado, y `calcular`, que hace el trabajo sin imprimir y es el que usa la clase `ComparadorMST` para medir tiempos. Ambas convierten cada relación en una arista con índices numéricos para trabajar más rápido.

### Prim

Hace crecer un solo árbol desde un vértice. En cada paso agrega la relación más liviana que conecta el árbol con un vértice nuevo. En el código usa una **lista de adyacencia** y una **cola de prioridad** (`PriorityQueue`) que entrega siempre la arista más liviana. Si al sacar una arista sus dos extremos ya están en el árbol, la descarta porque formaría un ciclo.

### Kruskal

Ordena todas las relaciones de menor a mayor peso y las recorre, aceptando cada una solo si no forma un ciclo. Para saberlo usa **Union-Find**: si los dos vértices están en conjuntos distintos ("islas"), la relación se acepta y los conjuntos se unen. Se detiene al llegar a $V - 1$ relaciones, sin revisar las que sobran.

### Decisiones comunes

- Si el grafo es **dirigido**, ambos avisan que no aplican, porque están definidos para grafos no dirigidos.
- Si el grafo **no es conexo**, se obtiene un bosque de expansión mínima y se indica cuántas componentes tiene.

# 3. Comparación

## 3.1. Comparación general

| | **Prim** | **Kruskal** |
|---|---|---|
| Idea | Un solo árbol que crece. | Ordena aristas y une islas. |
| Estructuras | Lista de adyacencia y cola de prioridad. | Lista de aristas y Union-Find. |
| Complejidad | $O(E \log V)$ | $O(E \log E)$ |

Las dos complejidades son del mismo orden, porque $\log E \leq 2 \log V$. En teoría solo se diferencian por un factor constante.

## 3.2. Resultados de la medición

`ComparadorMST` genera grafos aleatorios conexos, poco densos (unas 3 aristas por vértice) y densos (12,5 % de las aristas posibles). Cada caso se repite 5 veces y se promedia. La memoria es la cantidad de bytes asignados durante la ejecución, no el pico de uso.

| **Vértices** | **Aristas** | **Tipo** | **Prim (ms)** | **Kruskal (ms)** | **Prim (KB)** | **Kruskal (KB)** |
|---|---|---|---|---|---|---|
| 100 | 300 | Poco denso | 0,853 | 0,825 | 28 | 22 |
| 100 | 1237 | Denso | 1,114 | 1,134 | 97 | 75 |
| 500 | 1500 | Poco denso | 1,218 | 1,147 | 145 | 116 |
| 500 | 31187 | Denso | 10,345 | 9,309 | 2531 | 1511 |
| 1000 | 3000 | Poco denso | 0,886 | 0,803 | 298 | 242 |
| 1000 | 124875 | Denso | 38,701 | 30,090 | 9678 | 6374 |
| 2000 | 6000 | Poco denso | 1,504 | 1,048 | 612 | 461 |
| 2000 | 499750 | Denso | 329,085 | 106,116 | 34991 | 24169 |

En todos los casos ambos algoritmos obtuvieron el mismo peso total.

## 3.3. Casos donde son similares

- **Resultado:** siempre dan el mismo peso total.
- **Grafos poco densos:** los tiempos son casi iguales (entre 0,8 y 1,5 ms), incluso con 2000 vértices.
- **Casos pequeños:** con 100 vértices los tiempos difieren menos de 3 %, lo que es ruido de medición. Por eso en la fila de 100 vértices densos Prim aparece como ganador por solo 0,02 ms, pero debe leerse como empate.
- **Grafos no conexos y dirigidos:** se comportan igual en ambos (bosque y aviso, respectivamente).

## 3.4. Casos donde son diferentes

En los grafos grandes y densos Kruskal gana de forma consistente, y la diferencia crece con el tamaño:

| **Caso denso** | **Tiempo Prim / Kruskal** | **Memoria Prim / Kruskal** |
|---|---|---|
| 500 vértices | 1,1 veces | 1,7 veces |
| 1000 vértices | 1,3 veces | 1,5 veces |
| 2000 vértices | 3,1 veces | 1,4 veces |

Las razones más probables, que no se midieron por separado, son:

- Kruskal **se detiene al elegir $V-1$ relaciones**, mientras que Prim hace pasar casi todas las aristas por la cola.
- Ordenar un arreglo una sola vez suele ser más barato que muchas inserciones en una cola de prioridad.
- Prim guarda más estructuras (cada arista aparece dos veces en la lista de adyacencia), por eso usa más memoria.

### 3.5. ¿Cuándo usar cada uno?

- **Kruskal:** cuando el grafo está como lista de aristas, las aristas ya vienen ordenadas, o el grafo es grande con listas como las de este programa.
- **Prim:** cuando el grafo está como matriz de adyacencia y es muy denso (allí cuesta $O(V^2)$), o cuando se quiere un árbol que crezca desde un vértice dado.
- **Cualquiera:** en grafos pequeños o poco densos.

El caso de la matriz de adyacencia viene de la teoría y no se midió, porque ambos algoritmos se implementaron con listas.

## 4. Conclusión

Prim y Kruskal dan siempre el mismo peso total y son casi iguales en grafos pequeños o poco densos. En grafos grandes y densos, Kruskal fue más rápido (hasta 3,1 veces) y usó menos memoria en esta implementación. No significa que Kruskal sea mejor en general: los tiempos dependen del equipo y de cómo se guarda el grafo, y con una matriz de adyacencia Prim podría ser la mejor opción.